InfraPilot AI

System Architecture — High & Low Level Design

InfraPilot AI is an AI-powered IaaS cloud infrastructure automation platform. Users visually design AWS architecture with a node-based builder, connect an AWS account, sync live account insights, generate Terraform, and deploy supported resources.

Technology stack

React + TypeScript	Vite	React Flow	Zustand	Tailwind CSS
Node.js / Express	MongoDB + Mongoose	JWT Auth	AWS SDK v3	Terraform CLI

Architecture layers

Frontend — Vite / React — src/

Landing page	Auth page	Dashboard shell	Visual builder	Deploy modal
Zustand diagram store	AWS service catalog	Diagram validator	Terraform export util	

Backend — Node.js / Express — IAAS backend/src/

Auth API	AWS API	Diagram API	Deploy API	Agent API
Terraform generator	Terraform runner	AWS live sync	Role credentials	

Storage — MongoDB Atlas + local Terraform state

Users	Workspaces	AWS accounts	Diagrams	Deployments	Audit logs

AWS cloud — Live sync + Terraform deployment target

STS role assumption	Cost Explorer	EC2 / Lambda	S3 / RDS / DynamoDB	CW / CloudTrail

VPC	EC2	Lambda	API Gateway	S3	DynamoDB	SQS	SNS	EventBridge

Frontend components

Landing page	Public SaaS marketing page — src/landing/

Auth page	Login + registration forms; stores JWT in localStorage (infra-auth-token)
Dashboard shell	DashboardShell.tsx — AWS insights, connect-AWS form, deployments list
Visual builder	Canvas.tsx + React Flow; drag-drop AWS service nodes, edge connections, undo/redo, export
Deploy modal	DeploymentModal.tsx — account selector, Terraform preview, log polling
Zustand store	diagramStore.ts — nodes, edges, history, clipboard, validation issues
AWS service catalog	awsServices.ts — service IDs, icons, ports, Terraform resource types, default config
Validation util	validate.ts — pre-deployment diagram checks; surfaces errors in builder UI
Deployment API util	deploymentApi.ts — createCanvasDeployment(), getDeployment(), applyDeployment()

Backend API routes & services

POST /auth/register	Create workspace + first user; hash password with bcrypt; issue JWT
POST /auth/login	Validate credentials; return access token + refresh token
GET /aws/accounts	List connected AWS accounts for workspace
POST /aws/accounts	Connect account — validate via STS identity, run live sync
POST /aws/accounts/:id/sync	Re-sync all supported AWS services for account
GET /aws/insights	Return latest syncSummary for dashboard
GET /diagrams	List workspace diagrams
POST /diagrams	Save / update diagram nodes + edges
POST /deployments/from-canvas	Save diagram → validate → generate Terraform → run runner
GET /deployments/:id	Return deployment status + logs (polled by frontend)
POST /deployments/:id/apply	Trigger Terraform apply for planned deployment

Terraform generator — supported resource types

apigw	**lambda**	**dynamodb**	**s3**	**sqs**	**sns**	**eventbridge**

cloudwatch	**iam**	**secrets**	**vpc**	**ec2**

The generator produces an AWS provider block, optional AMI data source for EC2, Lambda execution role + policy attachment, stub zip reference, API Gateway-Lambda integrations, and EventBridge-Lambda targets. Unsupported nodes are emitted as comments.

Deployment pipeline — step by step

1. Frontend submit	DeploymentModal posts {name, awsAccountId, activeRegion, nodes, edges, autoApply}
2. Save diagram	Backend creates / updates a Diagram document from canvas nodes + edges
3. Build plan	buildDeploymentPlan() — validate diagram, count resources, generate Terraform
4. Create deployment	Deployment record saved with status=draft; stores terraform HCL, issues, plan
5. Terraform init	Runner writes main.tf + lambda_stub.zip to isolated work dir; runs terraform init
6. Terraform plan	terraform plan -out=tfplan; output appended to deployment logs
7. Terraform apply	terraform apply -auto-approve tfplan; status set to deploying → deployed/failed
8. Poll logs	Frontend polls GET /deployments/:id every few seconds; shows live log output

MongoDB data model

User	Identity, bcrypt password, role, workspace ref, status
Workspace	Tenant boundary — scopes all diagrams, accounts, deployments
AwsAccount	accountId, roleArn, externalId, defaultRegion, syncSummary, lastSyncAt
Diagram	nodes[], edges[], activeRegion, validationIssues[], tags
Deployment	status, terraform (HCL), logs[], plan, awsAccount ref, startedAt/finishedAt
AgentConversation	AI agent conversation context per workspace
AuditLog	Operational + security events (deployment create, apply, queue)

Security design

Authentication	JWT access tokens + refresh tokens; bcrypt password hashing
Authorization	Role-based (roles.js hierarchy); requireAuth + authorize(role) middleware
Data isolation	All queries scoped by workspace; no cross-tenant leakage
Transport	Helmet headers, CORS, rate limiting, request size limits
AWS access	STS role assumption with external ID; no hardcoded keys in source
Terraform state	Local state gitignored; production must use encrypted remote S3 state

Current limitations

Terraform execution	Runs in-process on backend; no isolated worker queue
State management	Local file state — no S3 remote state or DynamoDB locking
Destroy / rollback	Not implemented; no drift detection
Resource coverage	Subset of AWS services; advanced config options limited
AI agent	Utility-driven responses; not connected to a production LLM workflow

Planned enhancements

Remote TF state	**Destroy workflow**	**Approval gates**	**Team SSO**	**Background queue**
Full AWS coverage	**OpenAI agent actions**	**Drift detection**	**Audit exports**	**OTel tracing**
